

INTERNAL ENGINEERING DOCUMENTATION

Object Storage Service Codebase Guide

A complete, ground-up walkthrough of the architecture, layers, request flow, design decisions, and test strategy — written so you can answer any interview question about this project.

ASP.NET Core

Clean Architecture

MinIO (S3-compatible)

PostgreSQL

EF Core

Docker

OpenTelemetry

Document generated from the ObjectStorage codebase · Keep it updated as the project evolves

Table of Contents

1	What This Project Is
2	Architecture Overview
3	The Four Layers
4	Domain Layer — The Rules
5	Application Layer — The Orchestration
6	Infrastructure Layer — The Wiring
7	API Layer — The HTTP Interface
8	How a Request Flows End to End
9	Key Design Decisions
10	Configuration
11	Docker Setup
12	Testing Guide
13	Interview Cheat Sheet

1. What This Project Is

This is an **internal Object Storage Service** — a company-wide API that other backend services use to store and retrieve files. It wraps **MinIO** (an S3-compatible object store) behind a clean HTTP API so that consuming services never depend on MinIO directly.

Why not just use MinIO directly?

- Centralized authentication and authorization (each service can only access its own files)
- Consistent API contract across the company
- Audit logging of every operation
- Validation, idempotency, and error handling in one place
- If we ever switch storage backends, only this service changes

The architecture

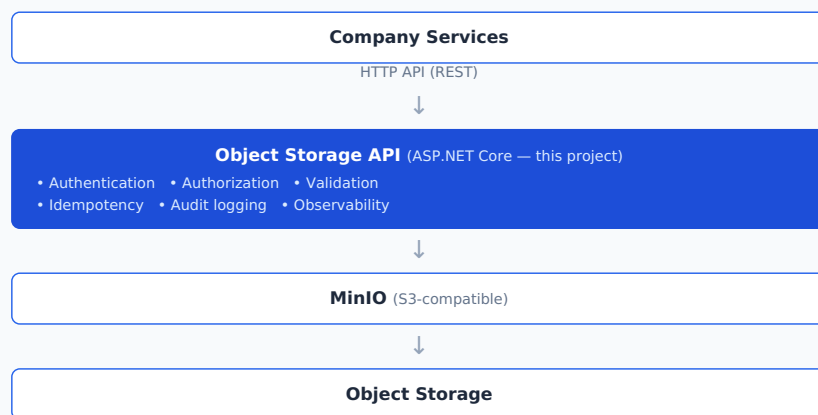


Figure 1 — Consuming services never talk to MinIO directly

2. Architecture Overview

The project uses **Clean Architecture** (also called Onion Architecture). The key rule: **dependencies point inward**.

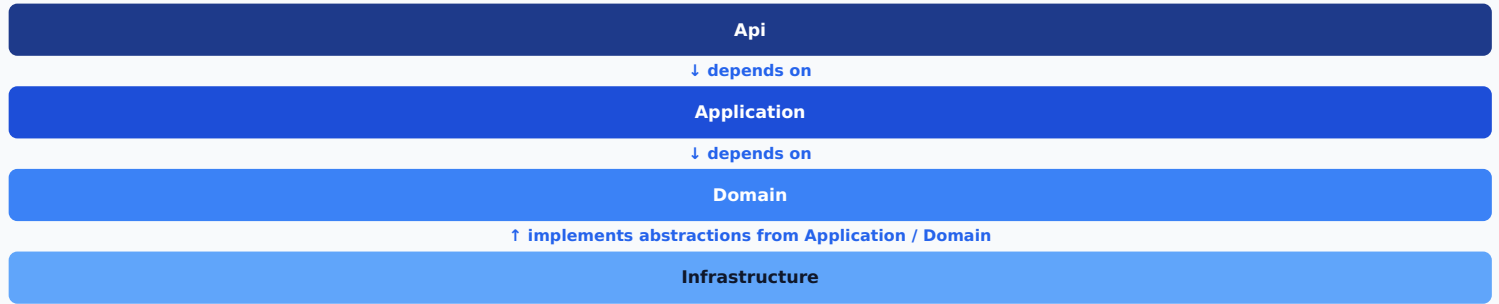


Figure 2 — Api → Application → Domain, with Infrastructure implementing the abstractions

What this means in practice

- **Domain** knows about nothing — it defines the core rules and entities
- **Application** knows about Domain — it orchestrates business logic
- **Infrastructure** knows about Application and Domain — it implements interfaces using real databases, MinIO, etc.
- **Api** knows about everything — it wires everything together and exposes HTTP endpoints

Why this matters: You can swap Infrastructure implementations (e.g., replace MinIO with S3, replace PostgreSQL with SQL Server) without changing Application or Domain code. The Application layer only depends on interfaces (abstractions), not concrete implementations.

3. The Four Layers

Project Structure

```
src/
├─ ObjectStorage.Domain/      ← Core rules, zero dependencies
├─ ObjectStorage.Application/ ← Business logic, depends only on Domain
├─ ObjectStorage.Infrastructure/ ← Real implementations (MinIO, EF Core, etc.)
└─ ObjectStorage.Api/        ← HTTP endpoints, wires everything together

tests/
├─ ObjectStorage.UnitTests/    ← Fast tests, no real database
└─ ObjectStorage.IntegrationTests/ ← Tests with real PostgreSQL + MinIO
```

4. Domain Layer — The Rules

The Domain layer has **zero external dependencies**. It defines what the system *is*, not *how* it works.

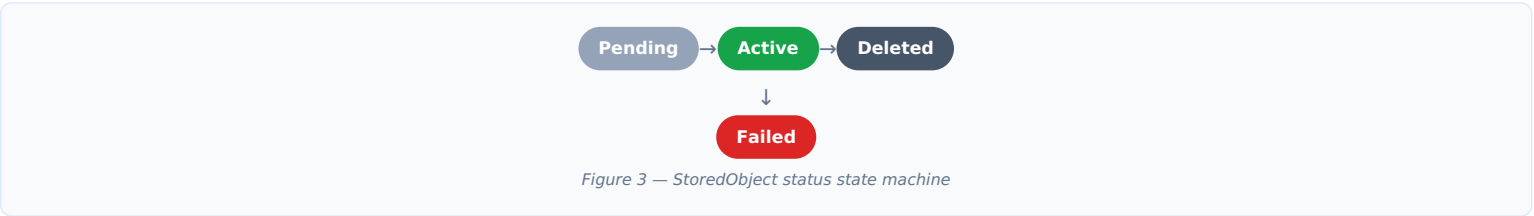
4.1 Entities

StoredObject (src/ObjectStorage.Domain/Entities/StoredObject.cs)

This is the **metadata record** for every uploaded file. It does NOT contain the file contents — those live in MinIO. This lives in PostgreSQL.

Property	Type	Purpose
<code>Id</code>	<code>string</code>	Public API identifier (ULID, e.g. <code>01K...</code>)
<code>ObjectKey</code>	<code>string</code>	Internal MinIO path (e.g. <code>documents/invoices/01K.../policy.pdf</code>)
<code>FileName</code>	<code>string</code>	Original filename the client uploaded
<code>ContentType</code>	<code>string</code>	MIME type (e.g. <code>application/pdf</code>)
<code>Size</code>	<code>long</code>	File size in bytes
<code>CreatedAt</code>	<code>DateTimeOffset</code>	When it was uploaded
<code>Status</code>	<code>StoredObjectStatus</code>	Current state in the lifecycle

Status lifecycle



- **Pending** — metadata created, upload to MinIO in progress
- **Active** — upload succeeded, object is available
- **Failed** — upload to MinIO failed, metadata is stale
- **Deleted** — soft-deleted (metadata kept for reconciliation)

The `MarkActive()` , `MarkFailed()` , `MarkDeleted()` methods are the only way to change status — this enforces the state machine.

ServiceIdentity (src/ObjectStorage.Domain/Entities/ServiceIdentity.cs)

Represents **who is calling the API**. Each company service gets an API key that maps to a `ServiceIdentity` with a name and allowed path prefixes.

```
// DocumentService can only access objects under "documents/"
new ServiceIdentity("DocumentService", ["documents/"]);

// Checks if an object key belongs to this service
identity.IsAuthorizedForObject("documents/invoices/01K.../policy.pdf"); // true
identity.IsAuthorizedForObject("users/avatars/01K.../photo.png");      // false
```

4.2 Interfaces (Abstractions)

These define contracts that Infrastructure must implement. The Application layer programs against these, never against concrete classes.

Interface	File	What it abstracts
<code>IObjectStorage</code>	<code>Domain/Interfaces/IObjectStorage.cs</code>	Upload/download/delete/presigned URLs (MinIO behind this)
<code>IServiceIdentity</code>	<code>Domain/Interfaces/IServiceIdentity.cs</code>	Who is calling (name + allowed prefixes)
<code>IServiceResolver</code>	<code>Domain/Interfaces/IServiceResolver.cs</code>	API key → <code>ServiceIdentity</code> lookup

`IObjectStorage` — the core abstraction

```
public interface IObjectStorage
{
    Task<ObjectMetadata> UploadAsync(Stream content, string objectKey, string contentType, long contentLength, CancellationToken ct);
    Task<Stream> DownloadAsync(string objectKey, CancellationToken ct);
    Task DeleteAsync(string objectKey, CancellationToken ct);
    Task<bool> ExistsAsync(string objectKey, CancellationToken ct);
    Task<string> CreatePresignedUrlAsync(string objectKey, TimeSpan expiration, CancellationToken ct);
}
```

Key insight: Application code never sees MinIO SDK types. It only sees `Stream` , `string` , and our own `ObjectMetadata` value object. This is how we hide MinIO from consuming services.

4.3 Value Objects

`ObjectMetadata` (src/ObjectStorage.Domain/ValueObjects/ObjectMetadata.cs)

A simple record returned by the storage layer after upload:

```
public record ObjectMetadata(string ObjectKey, long Size, string ContentType);
```

4.4 Exceptions

All domain exceptions inherit from `ObjectStorageException`. They map to specific HTTP status codes:

Exception	HTTP Status	When
<code>ObjectNotFoundException</code>	404	Object ID doesn't exist
<code>InvalidRequestException</code>	400	Bad input (empty file, bad expiration)
<code>CategoryNotAllowedException</code>	400	Category not in whitelist
<code>FileTooLargeException</code>	413	File exceeds <code>MaxFileSizeBytes</code>
<code>UnsupportedContentTypeException</code>	415	MIME type not in allowlist
<code>UnauthorizedAccessException</code>	401	Missing or invalid API key
<code>ForbiddenAccessException</code>	403	Service trying to access another service's object
<code>StorageUnavailableException</code>	503	MinIO or PostgreSQL is down

Why custom exceptions? They let us map domain concepts to HTTP responses centrally in `Program.cs` without scattering try/catch blocks everywhere.

5. Application Layer — The Orchestration

The Application layer contains the business logic. It depends only on Domain (and `Microsoft.Extensions.Options` for configuration).

5.1 Configuration

`StorageOptions` (src/ObjectStorage.Application/Configuration/StorageOptions.cs)

Strongly-typed configuration — no magic strings:

```
{
  "Storage": {
    "DefaultService": "objectstorage",
    "AllowedCategories": ["documents", "images", "invoices", "backups"],
    "MaxFileSizeBytes": 104857600,
    "AllowedContentTypes": ["application/pdf", "image/png", "image/jpeg"],
    "MaxPresignedUrlExpirationSeconds": 604800
  }
}
```

- `AllowedCategories` — clients must specify one of these when uploading
- `AllowedContentTypes` — MIME type whitelist; use `["*"]` to accept all
- `MaxFileSizeBytes` — default 100 MB
- `MaxPresignedUrlExpirationSeconds` — default 7 days

5.2 Interfaces (Application-level abstractions)

Interface	Purpose
<code>IObjectMetadataRepository</code>	CRUD for <code>StoredObject</code> metadata in PostgreSQL
<code>IIdempotencyRepository</code>	CRUD for idempotency records
<code>IObjectIdGenerator</code>	Generates unique object IDs (ULID)
<code>IObjectKeyGenerator</code>	Generates MinIO object keys from service/category/id/filename
<code>IAuditLogger</code>	Writes audit log entries

Why interfaces? So we can swap implementations (e.g., use in-memory repositories for unit tests) and so the Application layer doesn't know about PostgreSQL or MinIO.

5.3 Entities

`IdempotencyRecord` (src/ObjectStorage.Application/Entities/IdempotencyRecord.cs)

Tracks upload idempotency:

```
public sealed class IdempotencyRecord
{
    public string Key { get; set; } // The idempotency key from the client
    public string Status { get; set; } // "InProgress", "Completed", "Failed"
    public string? ObjectId { get; set; } // The object ID if completed
    public DateTimeOffset CreatedAt { get; set; }
    public DateTimeOffset? CompletedAt { get; set; }
}
```

`AuditLogEntry` (src/ObjectStorage.Application/Entities/AuditLogEntry.cs)

Audit trail for every operation:

```
public sealed class AuditLogEntry
{
    public long Id { get; set; }
    public DateTimeOffset Timestamp { get; set; }
    public string ServiceName { get; set; } // Which service called
    public string Operation { get; set; } // Upload, Download, Delete, etc.
    public string? ObjectId { get; set; } // Which object
    public string Result { get; set; } // "Success" or "Failure"
    public string CorrelationId { get; set; } // Request tracking ID
    public string? IpAddress { get; set; }
    public string? Metadata { get; set; } // Extra context (category, size, etc.)
}
```

5.4 `ObjectService` — The Brain

`src/ObjectStorage.Application/Services/ObjectService.cs` — this is the **most important class** in the project. It orchestrates everything.

Constructor dependencies


```
public ObjectService(
    IObjectStorage objectStorage,           // MinIO (via interface)
    IObjectMetadataRepository metadataRepo, // PostgreSQL (via interface)
    IObjectIdGenerator idGenerator,         // ULID generator
    IObjectKeyGenerator keyGenerator,       // Key strategy
    IOptions<StorageOptions> storageOptions, // Configuration
    IIdempotencyRepository? idempotencyRepo = null, // Optional
    IAuditLogger? auditLogger = null)         // Optional
```

Upload flow (the most complex operation)

- Validate request (empty file? allowed category? size limit? content type?)
- Check idempotency key if provided
InProgress → reject (400) | Completed → return existing result (no re-upload) | Failed → remove record, allow retry
- Create new IdempotencyRecord as InProgress (if key provided)
- Generate object ID (ULID) and object key (hierarchical path)
- Verify authorization (service can only upload to its own prefix)
- Insert metadata as "Pending" in PostgreSQL
- Stream file to MinIO
- On success: mark metadata "Active", mark idempotency "Completed"
- On failure: mark metadata "Failed", mark idempotency "Failed"
- Audit log the result

Why Pending → Active? This is the consistency strategy. Metadata is the source of truth. If MinIO succeeds but the API crashes before marking Active, the cleanup job will purge the stale Pending row. No distributed transaction needed.

Download flow

- Load metadata by ID (404 if not found)
- Verify authorization (service owns this object?)
- Stream from MinIO to client
- Audit log

Delete flow

- Load metadata by ID (404 if not found)
- Verify authorization
- Delete from MinIO
- Soft-delete metadata (set status to Deleted)
- Audit log

Presigned URL flow

- Validate expiration (> 0 and ≤ max)
- Load metadata by ID
- Verify authorization
- Generate presigned URL via MinIO SDK
- Return URL + server-computed expiresAt
- Audit log

6. Infrastructure Layer — The Wiring

Infrastructure implements the abstractions defined in Domain and Application using real technologies.

6.1 MinIO Integration

MinioObjectStorage (src/ObjectStorage.Infrastructure/MinIO/MinioObjectStorage.cs)

Implements **IObjectStorage** using the MinIO .NET SDK v7.

Key behaviors

- **Lazy bucket creation** — ensures the bucket exists on first use (guarded by **SemaphoreSlim** for thread safety)
- **Download copies to MemoryStream** — MinIO SDK v7 uses a callback model; the stream is fully read before the method returns. Large files use presigned URLs instead.
- **Every method is instrumented** — wrapped in **ActivitySource.StartActivity()** for distributed tracing, records metrics on success/failure
- **All MinIO exceptions are mapped** through **MinioExceptionMapper** — SDK types never leak past this class

MinioExceptionMapper (src/ObjectStorage.Infrastructure/MinIO/MinioExceptionMapper.cs)

Translates MinIO SDK exceptions to domain exceptions:

SDK Exception	Becomes
ObjectNotFoundException	ObjectNotFoundException (domain)
BucketNotFoundException	StorageUnavailableException
AccessDeniedException	StorageUnavailableException
Other MinioException	StorageUnavailableException
Domain exceptions	Passed through unchanged

MinioHealthCheck (src/ObjectStorage.Infrastructure/MinIO/MinioHealthCheck.cs)

Probes MinIO by calling **PingAsync** (checks bucket exists). Used by **/health/ready**.

6.2 Persistence (PostgreSQL + EF Core)

ObjectStorageDbContext (src/ObjectStorage.Infrastructure/Persistence/ObjectStorageDbContext.cs)

Entity Framework Core DbContext with four tables:

DbSet	Entity	Purpose
StoredObjects	StoredObject	File metadata
RegisteredServices	RegisteredService	API key → service mapping
IdempotencyRecords	IdempotencyRecord	Upload idempotency tracking
AuditLogEntries	AuditLogEntry	Audit trail

Entity configuration includes indexes on frequently queried columns (ObjectKey, ApiKeyHash, Timestamp, CorrelationId).

EfObjectMetadataRepository (src/ObjectStorage.Infrastructure/Persistence/EfObjectMetadataRepository.cs)

Implements **IObjectMetadataRepository**. Key behavior:

- **GetByIdAsync** only returns **Active** objects (soft-delete filtering)
- **DeleteAsync** sets status to **Deleted** (soft delete, not physical)

EfIdempotencyRepository (src/ObjectStorage.Infrastructure/Persistence/EfIdempotencyRepository.cs)

Implements **IIdempotencyRepository** — straightforward CRUD on the **IdempotencyRecords** table.

EfRegisteredServiceRepository (src/ObjectStorage.Infrastructure/Persistence/EfRegisteredServiceRepository.cs)

Looks up registered services by API key hash.

EfAuditLogRepository (src/ObjectStorage.Infrastructure/Persistence/EfAuditLogRepository.cs)

Implements both **IAuditLogger** and a repository — writes **AuditLogEntry** rows to PostgreSQL.

PostgresHealthCheck (src/ObjectStorage.Infrastructure/Persistence/PostgresHealthCheck.cs)

Runs **SELECT 1** against PostgreSQL. Used by **/health/ready**.

6.3 Authentication

ServiceResolver (src/ObjectStorage.Infrastructure/Services/ServiceResolver.cs)

Resolves an API key to a **ServiceIdentity**:

- Hash the key with SHA-256
- Look up `RegisteredService` in PostgreSQL by hash
- If found and active, return a `ServiceIdentity` with the service's name and allowed prefixes
- If not found, return null (→ 401)

Why hash? We never store raw API keys. Only the SHA-256 hash lives in the database. Even if the DB is compromised, the keys are safe.

`ApiKeyMiddleware` (src/ObjectStorage.Infrastructure/Middleware/ApiKeyMiddleware.cs)

Runs before every request (except health endpoints):

- Extract `X-Correlation-Id` header (or generate one)
- Extract `X-API-Key` header
- If no key → 401
- In Development mode: `dev-bypass-key` skips DB lookup (uses config default service)
- Resolve key via `ServiceResolver`
- If invalid → 401
- Store `IServiceIdentity` on `HttpContext.Items` for controllers to use

`SecurityHeadersMiddleware` (src/ObjectStorage.Infrastructure/Middleware/SecurityHeadersMiddleware.cs)

Adds standard security headers to every response:

`X-Content-Type-Options: nosniff` `X-Frame-Options: DENY` `X-XSS-Protection: 1; mode=block` `Cache-Control: no-store, no-cache, must-revalidate`
`Referrer-Policy: no-referrer` `Strict-Transport-Security (HTTPS only)`

6.4 Object ID and Key Generation

`UlidObjectIdGenerator` (src/ObjectStorage.Infrastructure/Services/UlidObjectIdGenerator.cs)

Generates 26-character Crockford Base32 ULIDs:

- Time-sortable (lexicographic ordering = chronological ordering)
- Non-predictable (unlike sequential IDs)
- URL-safe (no special characters)
- Example: `01K2G5H8R9N3X7B4T6Y1V8J2Q`

`HierarchicalObjectKeyGenerator` (src/ObjectStorage.Infrastructure/Services/HierarchicalObjectKeyGenerator.cs)

Generates MinIO object keys in the format:

```
{service}/{category}/{objectId}/{sanitized-filename}
```

Example: `documents/invoices/01K2G5H8R9N3X7B4T6Y1V8J2Q/policy.pdf`

Sanitization: removes path separators, control characters, trims whitespace.

6.5 Telemetry

`ObjectStorageMetrics` (src/ObjectStorage.Infrastructure/Telemetry/ObjectStorageTelemetry.cs)

OpenTelemetry metrics:

Metric	Type	What it measures
<code>objectstorage.upload.size_bytes</code>	Histogram	Upload file sizes
<code>objectstorage.download.size_bytes</code>	Histogram	Download file sizes
<code>objectstorage.operations.total</code>	Counter	Operations by type + result
<code>objectstorage.auth_failures.total</code>	Counter	Auth failures by exception type

`ObjectStorageTracing`

OpenTelemetry distributed tracing via `ActivitySource("ObjectStorage.MinIO")`. Each MinIO operation creates a span with semantic tags.

6.6 DI Registration

`ServiceCollectionExtensions.AddObjectStorage()` (src/ObjectStorage.Infrastructure/Configuration/ServiceCollectionExtensions.cs)

One call registers everything:

```
services.AddObjectStorage(configuration);
```

This binds configuration, registers all implementations, and sets up EF Core with PostgreSQL.

7. API Layer — The HTTP Interface

7.1 Program.cs — The Entry Point

src/ObjectStorage.Api/Program.cs wires the entire application:

- Register services (DI container)
- Configure OpenTelemetry
- Apply EF Core migrations (schema creation)
- Exception handler (maps domain exceptions → ProblemDetails)
- Security headers middleware
- API key authentication middleware
- Status code pages
- Controllers
- Health endpoints

The **exception handler** is centralized — it catches all domain exceptions and maps them to `ProblemDetails` JSON responses. This means controllers don't need try/catch blocks.

7.2 ObjectsController — The HTTP Endpoints

src/ObjectStorage.Api/Controllers/ObjectsController.cs exposes 5 endpoints:

Method	Route	Purpose
POST	/api/v1/objects	Upload a file (multipart form)
GET	/api/v1/objects/{id}/download	Download file contents
GET	/api/v1/objects/{id}/metadata	Get file metadata
DELETE	/api/v1/objects/{id}	Delete a file
POST	/api/v1/objects/{id}/presigned-url	Generate a presigned URL

Each endpoint

1. Extracts `IServiceIdentity` from `HttpContext.Items` (set by middleware)
2. Extracts correlation ID
3. Calls `ObjectService` method
4. Returns appropriate HTTP response

7.3 Contracts (DTOs)

Contract	Purpose
ObjectMetadataResponse	Returned by upload, metadata, and list endpoints
PresignedUrlRequest	Body for presigned URL generation (<code>expirationSeconds</code>)
PresignedUrlResponse	Returned with presigned URL + <code>expiresAt</code>

7.4 Health Endpoints

Endpoint	Purpose
/health/live	Liveness — always 200 if process is running (no dependency checks)
/health/ready	Readiness — checks MinIO + PostgreSQL connectivity
/health	Full health report

8. How a Request Flows End to End

Let's trace a **file upload** through the entire stack:

- Client sends `POST /api/v1/objects` with file + category + `X-API-Key` header
- `SecurityHeadersMiddleware` adds security headers to response
- `ApiKeyMiddleware` :
Reads X-API-Key header → Hashes key with SHA-256 → Looks up RegisteredService in PostgreSQL → Creates ServiceIdentity (name + allowed prefixes) → Stores it on `HttpContext.Items`
- `ObjectsController.Upload()` :
Gets ServiceIdentity from `HttpContext.Items` → Gets correlation ID → Reads Idempotency-Key header (optional) → Calls `ObjectService.UploadAsync()`
- `ObjectService.UploadAsync()` :
Validates: file not empty, category allowed, size OK, content type OK → Checks idempotency key if provided → Generates ULID (e.g. 01K2G5H8R9N3X7B4T6Y1V8J2Q) → Generates object key: `documents/images/01K2G5H8R9N3X7B4T6Y1V8J2Q/photo.png` → Verifies service is authorized for this prefix → Creates `StoredObject` metadata (Status = Pending) → Saves to PostgreSQL → Calls `MinioObjectStorage.UploadAsync()` (ensures bucket exists, streams file to MinIO, records metrics + tracing) → On success: marks metadata Active → Creates audit log entry → Returns `StoredObject`
- `ObjectsController` returns `201 Created` with `ObjectMetadataResponse`

Figure 4 — End-to-end upload request lifecycle

9. Key Design Decisions

9.1 No Distributed Transactions

Problem: Upload spans PostgreSQL (metadata) and MinIO (file). What if one succeeds and the other fails?

Solution: Metadata-first with status state machine.

1. Insert metadata as `Pending` (source of truth)
2. Upload to MinIO
3. On success → mark `Active`
4. On failure → mark `Failed`
5. Background cleanup job purges stale `Pending` rows

Tradeoff: Brief window where metadata exists but file isn't available. Acceptable for internal services.

9.2 Presigned URLs for Large Files

Problem: Proxying every byte through ASP.NET Core wastes memory and bandwidth.

Solution: For large files, clients get a presigned URL and download directly from MinIO.

Security tradeoff: Authorization is enforced at generation time, not consumption time. A leaked URL is valid until it expires. Acceptable with short expiry for internal services.

9.3 API Key Authentication (Not JWT)

Problem: Services need to authenticate, but we want simplicity.

Solution: SHA-256 hashed API keys in PostgreSQL. `IServiceIdentity` abstraction allows future migration to JWT/mTLS.

Tradeoff: Simpler than JWT but less feature-rich. Good enough for service-to-service communication.

9.4 Soft Delete + Cleanup Job

Problem: Delete must consider both PostgreSQL and MinIO. Either could fail.

Solution: Soft delete (set status to `Deleted`). Background job purges after 3 days. If MinIO delete fails, the object is still accessible until the job retries.

9.5 Idempotency via Header

Problem: Clients retry uploads on network failure, creating duplicates.

Solution: Optional `Idempotency-Key` header. Server tracks request state and returns stored result on retry.

10. Configuration

All configuration is strongly-typed and validated at startup.

`appsettings.json` **sections**

```
{
  "ConnectionStrings": {
    "Postgres": "Host=localhost;Port=5432;Database=objectstorage;..."
  },
  "MinIO": {
    "Endpoint": "localhost:9000",
    "AccessKey": "minioadmin",
    "SecretKey": "minioadmin-dev-only",
    "Bucket": "objectstorage",
    "UseSSL": false
  },
  "Storage": {
    "DefaultService": "objectstorage",
    "AllowedCategories": ["documents", "images", "invoices", "backups"],
    "MaxFileSizeBytes": 104857600,
    "AllowedContentTypes": ["application/pdf", "image/png", "image/jpeg", "text/plain", "application/octet-stream"],
    "MaxPresignedUrlExpirationSeconds": 604800
  },
  "Auth": {
    "HeaderName": "X-Api-Key",
    "SkipInDevelopment": true
  },
  "OpenTelemetry": {
    "Enabled": true,
    "ServiceName": "ObjectStorage.Api",
    "OtlpEndpoint": null,
    "ConsoleExporter": true
  }
}
```

In Docker: Environment variables override these using `__` separator (e.g., `MinIO__Endpoint`).

Secrets: Never committed to source control. Use environment variables, user secrets, or Docker secrets.

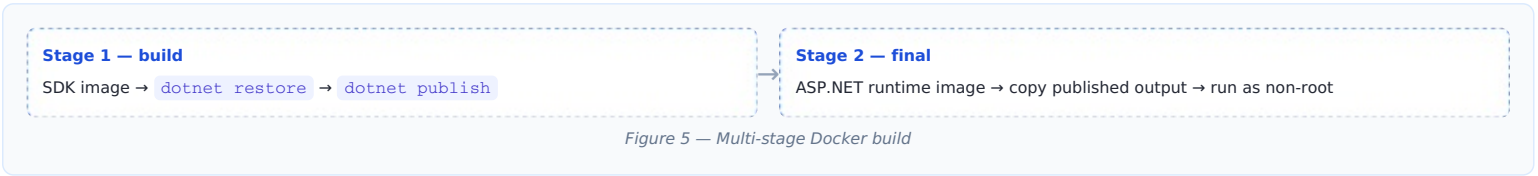
11. Docker Setup

docker/docker-compose.yml

Four services:

Service	Image	Purpose
postgres	postgres:16-alpine	Metadata database
minio	minio/minio:latest	Object storage
minio-init	minio/mc:latest	One-shot bucket creation
api	Built from Dockerfile	The API

Dockerfile (multi-stage build)



Running locally

```
docker compose -f docker/docker-compose.yml up --build
```

Then:

- API: <http://localhost:8080>
- MinIO Console: <http://localhost:9001> (minioadmin/minioadmin)
- PostgreSQL: <localhost:5432>

12. Testing Guide

12.1 Test Structure

```
tests/
├── ObjectStorage.UnitTests/           ← 79 tests, fast, no infrastructure
│   ├── Services/
│   │   ├── ObjectServiceTests.cs      ← Core upload/download/delete logic
│   │   ├── ObjectServiceValidationTests.cs ← Input validation rules
│   │   ├── ObjectServiceIdempotencyTests.cs ← Idempotency behavior
│   │   ├── ObjectServiceAuditTests.cs  ← Audit logging
│   │   ├── UlidObjectIdGeneratorTests.cs ← ULID contract
│   │   └── HierarchicalObjectKeyGeneratorTests.cs ← Key generation
│   ├── Persistence/
│   │   └── InMemoryObjectMetadataRepositoryTests.cs ← Repository contract
│   ├── Fakes/
│   │   ├── FakeObjectStorage.cs        ← In-memory IObjectStorage
│   │   ├── FakeIdempotencyRepository.cs ← In-memory idempotency
│   │   ├── FakeAuditLogger.cs          ← Records audit calls
│   │   └── FailingObjectStorage.cs      ← Throws on upload (for failure tests)
│   ├── MinioExceptionMapperTests.cs    ← Exception translation
│   ├── ObjectStorageTelemetryTests.cs  ← Metrics/tracing instruments
│   └── SecurityHeadersMiddlewareTests.cs ← Security headers
└── ObjectStorage.IntegrationTests/     ← 22 tests, real PostgreSQL + MinIO
    ├── Fixtures/
    │   └── TestcontainersFixture.cs    ← Starts real containers
    ├── IntegrationTestBase.cs          ← WebApplicationFactory setup
    └── ObjectStorageIntegrationTests.cs ← Full API tests
```

12.2 Running Tests

```
# Unit tests (fast, no dependencies)
dotnet test tests/ObjectStorage.UnitTests/

# Integration tests (needs Docker for Testcontainers)
dotnet test tests/ObjectStorage.IntegrationTests/

# All tests
dotnet test ObjectStorage.slnx
```

12.3 How Unit Tests Work

Unit tests use **fakes** (in-memory implementations) instead of real databases or MinIO.

Example from `ObjectServiceTests`:

```
// Arrange
var fakeStorage = new FakeObjectStorage();
var fakeRepo = new InMemoryObjectMetadataRepository();
var service = new ObjectService(fakeStorage, fakeRepo, ...);

// Act
var result = await service.UploadAsync(stream, identity, "documents", "test.pdf", "application/pdf", 100);

// Assert
Assert.Equal("documents/", result.ObjectKey);
Assert.Equal(StoredObjectStatus.Active, result.Status);
```

Why fakes instead of mocks?

- Fakes are real implementations (just in-memory)
- Tests verify behavior, not implementation details
- More robust against refactoring

12.4 How Integration Tests Work

Integration tests use **Testcontainers** to spin up real PostgreSQL and MinIO in Docker containers.

```
public class ObjectStorageIntegrationTests : IntegrationTestBase
{
    [Fact]
    public async Task Upload_Download_Delete_RoundTrip()
    {
        // Upload
        var uploadResponse = await Client.PostAsync("/api/v1/objects", multipartContent);
        Assert.Equal(HttpStatusCode.Created, uploadResponse.StatusCode);

        // Download
        var downloadResponse = await Client.GetAsync($"api/v1/objects/{id}/download");
        Assert.Equal(HttpStatusCode.OK, downloadResponse.StatusCode);
    }
}
```

```
// Delete
var deleteResponse = await Client.DeleteAsync($"api/v1/objects/{id}");
Assert.Equal(HttpStatusCode.NoContent, deleteResponse.StatusCode);
}
}
```

12.5 Key Test Scenarios to Know

Scenario	What It Tests	Expected Result
Upload → Metadata → Download → Delete	Full lifecycle	All succeed
Upload empty file	Validation	400 Bad Request
Upload unknown category	Category whitelist	400 Bad Request
Upload oversized file	Size limit	413 Payload Too Large
Upload unsupported MIME type	Content type whitelist	415 Unsupported Media Type
Download non-existent object	Not found	404 Not Found
Download with wrong API key	Authorization	403 Forbidden
Missing API key	Authentication	401 Unauthorized
Invalid API key	Authentication	401 Unauthorized
Presigned URL with zero expiration	Validation	400 Bad Request
Idempotent retry (same key)	Idempotency	Returns same object ID
MinIO failure during upload	Consistency	Metadata marked Failed
Security headers present	Security middleware	All headers set

12.6 Writing Your Own Tests

To test a new service method

1. Create a test in `tests/ObjectStorage.UnitTests/Services/`
2. Use `FakeObjectStorage` and `InMemoryObjectMetadataRepository` for isolation
3. Arrange → Act → Assert pattern
4. Test both success and failure paths

To test a new API endpoint

1. Add to `tests/ObjectStorage.IntegrationTests/ObjectStorageIntegrationTests.cs`
2. Extend `IntegrationTestBase` (gives you real containers)
3. Use `Client` (`HttpClient`) to call the endpoint
4. Assert status code and response body

13. Interview Cheat Sheet

Common Questions and Answers

Q: Why Clean Architecture?

A: Separation of concerns. Domain has zero dependencies, so business rules are pure and testable. You can swap PostgreSQL for SQL Server, or MinIO for S3, without changing business logic.

Q: How do you handle consistency between PostgreSQL and MinIO?

A: Metadata-first approach with a state machine (Pending → Active → Failed → Deleted). No distributed transactions. Cleanup job handles stale records.

Q: What's the upload flow?

A: Validate → Check idempotency → Generate IDs → Insert metadata as Pending → Stream to MinIO → Mark Active on success / Failed on failure → Audit log.

Q: How does authorization work?

A: API key → SHA-256 hash → lookup in PostgreSQL → ServiceIdentity with allowed prefixes. Every operation checks if the caller's prefix matches the object key.

Q: Why ULIDs instead of GUIDs?

A: ULIDs are time-sortable (lexicographic order = chronological order), URL-safe, and non-predictable. GUIDs are random and not sortable.

Q: Why soft delete?

A: Two-state delete (PostgreSQL + MinIO) can fail at either step. Soft delete keeps metadata for reconciliation. Background job purges after 3 days.

Q: How do presigned URLs work?

A: Client requests a presigned URL with expiration. API authorizes, then MinIO generates a signed URL. Client downloads directly from MinIO. Authorization is at generation time, not consumption time.

Q: What happens if MinIO is down during upload?

A: Metadata stays in Failed state. Idempotency record also marked Failed. Client can retry (idempotency key will allow fresh attempt). Cleanup job purges Failed metadata.

Q: How do you prevent path traversal attacks?

A: Object keys are generated server-side, never client-controlled. Client provides category + filename; the key generator sanitizes and constructs the path. `HierarchicalObjectKeyGenerator` strips path separators and control characters.

Q: What's the difference between liveness and readiness?

A: Liveness (`/health/live`) = "is the process running?" (always 200). Readiness (`/health/ready`) = "can it serve traffic?" (checks MinIO + PostgreSQL). Kubernetes uses liveness to restart pods, readiness to route traffic.

Q: How would you add rate limiting?

A: Use ASP.NET Core's built-in rate limiter. Per-service limiting via `IServiceIdentity.Name` as the partition key.

Q: How would you scale this horizontally?

A: The API is stateless — scale by adding instances behind a load balancer. PostgreSQL and MinIO handle their own scaling. Idempotency records in PostgreSQL prevent duplicate uploads across instances.

Q: What's the security model?

A: API key authentication (SHA-256 hashed), prefix-based authorization, security headers (nosniff, DENY frame, HSTS), read-only Docker filesystem, no-new-privileges, no secrets in source control.